Even with a sorted list, that means accessing `O(log(|E|))` elements for the operations above, and `O(|E|)` for listing all edges going into a vertex.

To no surprise, it turns out we can do better.

There are two main strategies to store a graph's edges:

- **Adjacency lists**: for each vertex `v`, we store a list of the edges `(v, u)`, where `v` is the source and `u`, the destination, is another vertex in `G`;
- **Adjacency matrix**: it's a `|V|×|V|` matrix, where the generic cell `(i,j)` contains the weight of the edge going from the `i`-th vertex to the `j`-th vertex (or `true`/`false`, or yet `1`/`0`, in case of un-weighted graphs, to state the presence/absence of an unweighted edge between those two vertices).

Before examining PROs and CONs of both strategies, let's illustrate them with an example: given the graph in figure 14.2, its adjacency lists representation is the following dictionary mapping vertices to lists of edges:

```
1 -> [(1,2), (1,3)]
2 -> [(2,4)]
3 -> []
4 -> []
```

The adjacency matrix representation, instead, is the following:

	V1	V2	V3	V4
V1				